

```
#!/usr/bin/env python3
"""Run the eval + improve loop until all pass or max iterations reached.

Combines run_eval.py and improve_description.py in a loop, tracking history
and returning the best description found. Supports train/test split to prevent
overfitting.
"""

import argparse
import json
import random
import sys
import tempfile
import time
import webbrowser
from pathlib import Path

from scripts.generate_report import generate_html
from scripts.improve_description import improve_description
from scripts.run_eval import find_project_root, run_eval
from scripts.utils import parse_skill_md


def split_eval_set(eval_set: list[dict], holdout: float, seed: int = 42) ->
tuple[list[dict], list[dict]]:
    """Split eval set into train and test sets, stratified by should_trigger."""
    random.seed(seed)

    # Separate by should_trigger
    trigger = [e for e in eval_set if e["should_trigger"]]
    no_trigger = [e for e in eval_set if not e["should_trigger"]]

    # Shuffle each group
    random.shuffle(trigger)
    random.shuffle(no_trigger)

    # Calculate split points
    n_trigger_test = max(1, int(len(trigger) * holdout))
    n_no_trigger_test = max(1, int(len(no_trigger) * holdout))

    # Split
    test_set = trigger[:n_trigger_test] + no_trigger[:n_no_trigger_test]
    train_set = trigger[n_trigger_test:] + no_trigger[n_no_trigger_test:]

    return train_set, test_set


def run_loop(
    eval_set: list[dict],
    skill_path: Path,
    description_override: str | None,
    num_workers: int,
    timeout: int,
    max_iterations: int,
    runs_per_query: int,

```

```

trigger_threshold: float,
holdout: float,
model: str,
verbose: bool,
live_report_path: Path | None = None,
log_dir: Path | None = None,
) -> dict:
    """Run the eval + improvement loop."""
    project_root = find_project_root()
    name, original_description, content = parse_skill_md(skill_path)
    current_description = description_override or original_description

    # Split into train/test if holdout > 0
    if holdout > 0:
        train_set, test_set = split_eval_set(eval_set, holdout)
        if verbose:
            print(f"Split: {len(train_set)} train, {len(test_set)} test
(holdout={holdout})", file=sys.stderr)
        else:
            train_set = eval_set
            test_set = []

    history = []
    exit_reason = "unknown"

    for iteration in range(1, max_iterations + 1):
        if verbose:
            print(f"\n{' '*60}", file=sys.stderr)
            print(f"Iteration {iteration}/{max_iterations}", file=sys.stderr)
            print(f"Description: {current_description}", file=sys.stderr)
            print(f"{' '*60}", file=sys.stderr)

        # Evaluate train + test together in one batch for parallelism
        all_queries = train_set + test_set
        t0 = time.time()
        all_results = run_eval(
            eval_set=all_queries,
            skill_name=name,
            description=current_description,
            num_workers=num_workers,
            timeout=timeout,
            project_root=project_root,
            runs_per_query=runs_per_query,
            trigger_threshold=trigger_threshold,
            model=model,
        )
        eval_elapsed = time.time() - t0

        # Split results back into train/test by matching queries
        train_queries_set = {q["query"] for q in train_set}
        train_result_list = [r for r in all_results["results"] if r["query"] in
train_queries_set]
        test_result_list = [r for r in all_results["results"] if r["query"] not in
train_queries_set]

```

```

    train_passed = sum(1 for r in train_result_list if r["pass"])
    train_total = len(train_result_list)
    train_summary = {"passed": train_passed, "failed": train_total - train_passed,
"total": train_total}
    train_results = {"results": train_result_list, "summary": train_summary}

    if test_set:
        test_passed = sum(1 for r in test_result_list if r["pass"])
        test_total = len(test_result_list)
        test_summary = {"passed": test_passed, "failed": test_total - test_passed,
"total": test_total}
        test_results = {"results": test_result_list, "summary": test_summary}
    else:
        test_results = None
        test_summary = None

    history.append({
        "iteration": iteration,
        "description": current_description,
        "train_passed": train_summary["passed"],
        "train_failed": train_summary["failed"],
        "train_total": train_summary["total"],
        "train_results": train_results["results"],
        "test_passed": test_summary["passed"] if test_summary else None,
        "test_failed": test_summary["failed"] if test_summary else None,
        "test_total": test_summary["total"] if test_summary else None,
        "test_results": test_results["results"] if test_results else None,
        # For backward compat with report generator
        "passed": train_summary["passed"],
        "failed": train_summary["failed"],
        "total": train_summary["total"],
        "results": train_results["results"],
    })

    # Write live report if path provided
    if live_report_path:
        partial_output = {
            "original_description": original_description,
            "best_description": current_description,
            "best_score": "in progress",
            "iterations_run": len(history),
            "holdout": holdout,
            "train_size": len(train_set),
            "test_size": len(test_set),
            "history": history,
        }
        live_report_path.write_text(generate_html(partial_output,
auto_refresh=True, skill_name=name))

    if verbose:
        def print_eval_stats(label, results, elapsed):
            pos = [r for r in results if r["should_trigger"]]
            neg = [r for r in results if not r["should_trigger"]]
            tp = sum(r["triggers"] for r in pos)
            pos_runs = sum(r["runs"] for r in pos)

```

```

        fn = pos_runs - tp
        fp = sum(r["triggers"] for r in neg)
        neg_runs = sum(r["runs"] for r in neg)
        tn = neg_runs - fp
        total = tp + tn + fp + fn
        precision = tp / (tp + fp) if (tp + fp) > 0 else 1.0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 1.0
        accuracy = (tp + tn) / total if total > 0 else 0.0
        print(f"{label}: {tp+tn}/{total} correct, precision={precision:.0%}
recall={recall:.0%} accuracy={accuracy:.0%} ({elapsed:.1f}s)", file=sys.stderr)
        for r in results:
            status = "PASS" if r["pass"] else "FAIL"
            rate_str = f"{r['triggers']}/{r['runs']}"
            print(f"    [{status}] rate={rate_str}
expected={r['should_trigger']}: {r['query'][:60]}", file=sys.stderr)

    print_eval_stats("Train", train_results["results"], eval_elapsed)
    if test_summary:
        print_eval_stats("Test ", test_results["results"], 0)

    if train_summary["failed"] == 0:
        exit_reason = f"all_passed (iteration {iteration})"
        if verbose:
            print(f"\nAll train queries passed on iteration {iteration}!",
file=sys.stderr)
            break

    if iteration == max_iterations:
        exit_reason = f"max_iterations ({max_iterations})"
        if verbose:
            print(f"\nMax iterations reached ({max_iterations}).", file=sys.stderr)
            break

    # Improve the description based on train results
    if verbose:
        print(f"\nImproving description...", file=sys.stderr)

    t0 = time.time()
    # Strip test scores from history so improvement model can't see them
    blinded_history = [
        {k: v for k, v in h.items() if not k.startswith("test_")}
        for h in history
    ]
    new_description = improve_description(
        skill_name=name,
        skill_content=content,
        current_description=current_description,
        eval_results=train_results,
        history=blinded_history,
        model=model,
        log_dir=log_dir,
        iteration=iteration,
    )
    improve_elapsed = time.time() - t0

```

```

        if verbose:
            print(f"Proposed ({improve_elapsed:.1f}s): {new_description}",
file=sys.stderr)

        current_description = new_description

# Find the best iteration by TEST score (or train if no test set)
if test_set:
    best = max(history, key=lambda h: h["test_passed"] or 0)
    best_score = f"{best['test_passed']}/{best['test_total']}"
else:
    best = max(history, key=lambda h: h["train_passed"])
    best_score = f"{best['train_passed']}/{best['train_total']}"

if verbose:
    print(f"\nExit reason: {exit_reason}", file=sys.stderr)
    print(f"Best score: {best_score} (iteration {best['iteration']})",
file=sys.stderr)

    return {
        "exit_reason": exit_reason,
        "original_description": original_description,
        "best_description": best["description"],
        "best_score": best_score,
        "best_train_score": f"{best['train_passed']}/{best['train_total']}",
        "best_test_score": f"{best['test_passed']}/{best['test_total']}" if test_set
else None,
        "final_description": current_description,
        "iterations_run": len(history),
        "holdout": holdout,
        "train_size": len(train_set),
        "test_size": len(test_set),
        "history": history,
    }

def main():
    parser = argparse.ArgumentParser(description="Run eval + improve loop")
    parser.add_argument("--eval-set", required=True, help="Path to eval set JSON file")
    parser.add_argument("--skill-path", required=True, help="Path to skill directory")
    parser.add_argument("--description", default=None, help="Override starting
description")
    parser.add_argument("--num-workers", type=int, default=10, help="Number of parallel
workers")
    parser.add_argument("--timeout", type=int, default=30, help="Timeout per query in
seconds")
    parser.add_argument("--max-iterations", type=int, default=5, help="Max improvement
iterations")
    parser.add_argument("--runs-per-query", type=int, default=3, help="Number of runs
per query")
    parser.add_argument("--trigger-threshold", type=float, default=0.5, help="Trigger
rate threshold")
    parser.add_argument("--holdout", type=float, default=0.4, help="Fraction of eval
set to hold out for testing (0 to disable)")
    parser.add_argument("--model", required=True, help="Model for improvement")

```

```

    parser.add_argument("--verbose", action="store_true", help="Print progress to
stderr")
    parser.add_argument("--report", default="auto", help="Generate HTML report at this
path (default: 'auto' for temp file, 'none' to disable)")
    parser.add_argument("--results-dir", default=None, help="Save all outputs
(results.json, report.html, log.txt) to a timestamped subdirectory here")
    args = parser.parse_args()

    eval_set = json.loads(Path(args.eval_set).read_text())
    skill_path = Path(args.skill_path)

    if not (skill_path / "SKILL.md").exists():
        print(f"Error: No SKILL.md found at {skill_path}", file=sys.stderr)
        sys.exit(1)

    name, _, _ = parse_skill_md(skill_path)

    # Set up live report path
    if args.report != "none":
        if args.report == "auto":
            timestamp = time.strftime("%Y%m%d_%H%M%S")
            live_report_path = Path(tempfile.gettempdir()) /
f"skill_description_report_{skill_path.name}_{timestamp}.html"
        else:
            live_report_path = Path(args.report)
            # Open the report immediately so the user can watch
            live_report_path.write_text("<html><body><h1>Starting optimization
loop...</h1><meta http-equiv='refresh' content='5'></body></html>")
            webbrowser.open(str(live_report_path))
    else:
        live_report_path = None

    # Determine output directory (create before run_loop so logs can be written)
    if args.results_dir:
        timestamp = time.strftime("%Y-%m-%d_%H%M%S")
        results_dir = Path(args.results_dir) / timestamp
        results_dir.mkdir(parents=True, exist_ok=True)
    else:
        results_dir = None

    log_dir = results_dir / "logs" if results_dir else None

    output = run_loop(
        eval_set=eval_set,
        skill_path=skill_path,
        description_override=args.description,
        num_workers=args.num_workers,
        timeout=args.timeout,
        max_iterations=args.max_iterations,
        runs_per_query=args.runs_per_query,
        trigger_threshold=args.trigger_threshold,
        holdout=args.holdout,
        model=args.model,
        verbose=args.verbose,
        live_report_path=live_report_path,

```

```
        log_dir=log_dir,
    )

    # Save JSON output
    json_output = json.dumps(output, indent=2)
    print(json_output)
    if results_dir:
        (results_dir / "results.json").write_text(json_output)

    # Write final HTML report (without auto-refresh)
    if live_report_path:
        live_report_path.write_text(generate_html(output, auto_refresh=False,
skill_name=name))
        print(f"\nReport: {live_report_path}", file=sys.stderr)

    if results_dir and live_report_path:
        (results_dir / "report.html").write_text(generate_html(output,
auto_refresh=False, skill_name=name))

    if results_dir:
        print(f"Results saved to: {results_dir}", file=sys.stderr)

if __name__ == "__main__":
    main()
```